

# API, REST и JSON

Связываем программы через понятный контракт.

## Главная идея

API - согласованный интерфейс, через который одна программа использует возможности другой. В веб-API клиент обращается к endpoint по HTTP.

```
{  
  "course": "IT Foundation",  
  "day": 23,  
  "passed": true  
}
```

# План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить  
термины

3

Разобрать код

4

Попробовать  
самому

5

Собрать мини-  
задачу

## Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

# Смысл темы простыми словами

Сначала бытовая идея, потом код.

## Идея 1

API - согласованный интерфейс, через который одна программа использует возможности другой. В веб-API клиент обращается к endpoint по HTTP.

## Идея 2

REST - распространённый стиль: ресурс имеет URL, а действие выражается методом. GET читает, POST создаёт, PUT или PATCH изменяет, DELETE удаляет.

## Идея 3

JSON часто служит телом запроса и ответа. Аутентификация подтверждает личность клиента, а token позволяет передавать доказательство доступа с запросом.

# Термины дня

Эти слова нужно узнавать в коде и документации.

## API

согласованный интерфейс взаимодействия программ

## endpoint

конкретный адрес операции веб-API

## REST

стиль проектирования API вокруг ресурсов и HTTP-методов

## JSON

текстовый формат структурированных данных

## authentication

проверка того, кто выполняет запрос

## token

строка, подтверждающая право клиента на запрос

# Код дня

Минимальный пример без лишней сложности.



```
{  
  "course": "IT Foundation",  
  "day": 23,  
  "passed": true  
}
```

## Что ищем

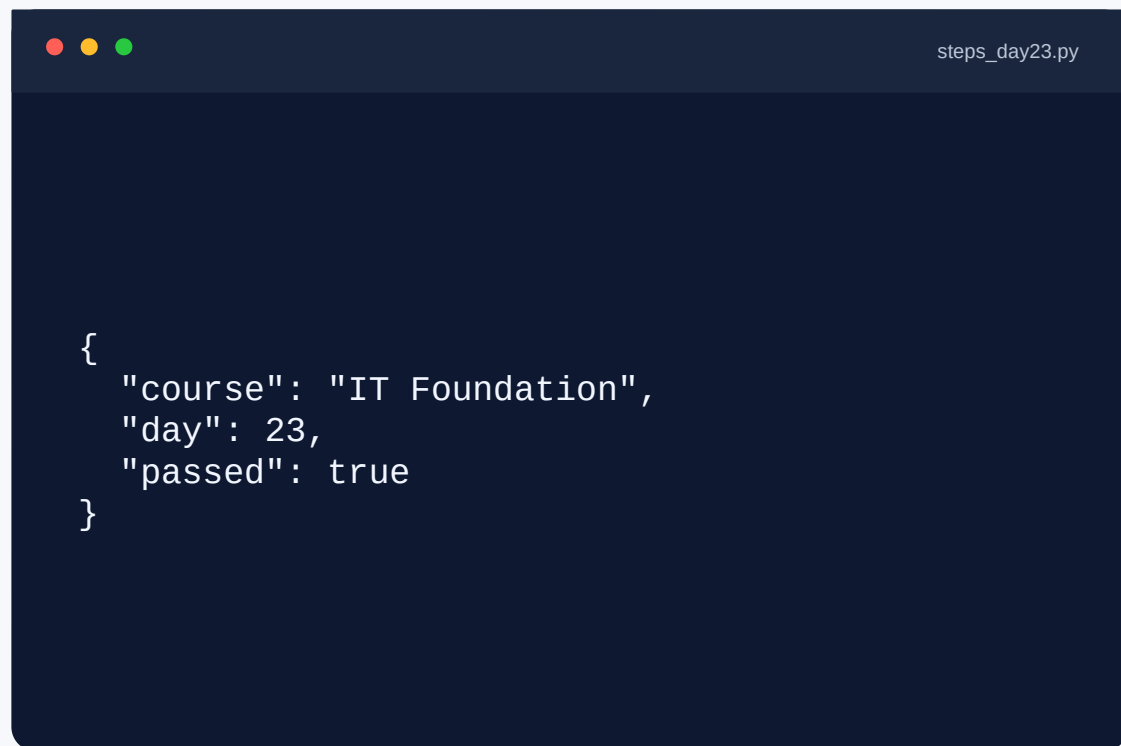
Где данные? Где действие? Где результат?

## Главный вопрос

Можешь объяснить каждую строку обычным языком?

# Разбор кода по шагам

Не переписываем вслепую.



```
{  
  "course": "IT Foundation",  
  "day": 23,  
  "passed": true  
}
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.

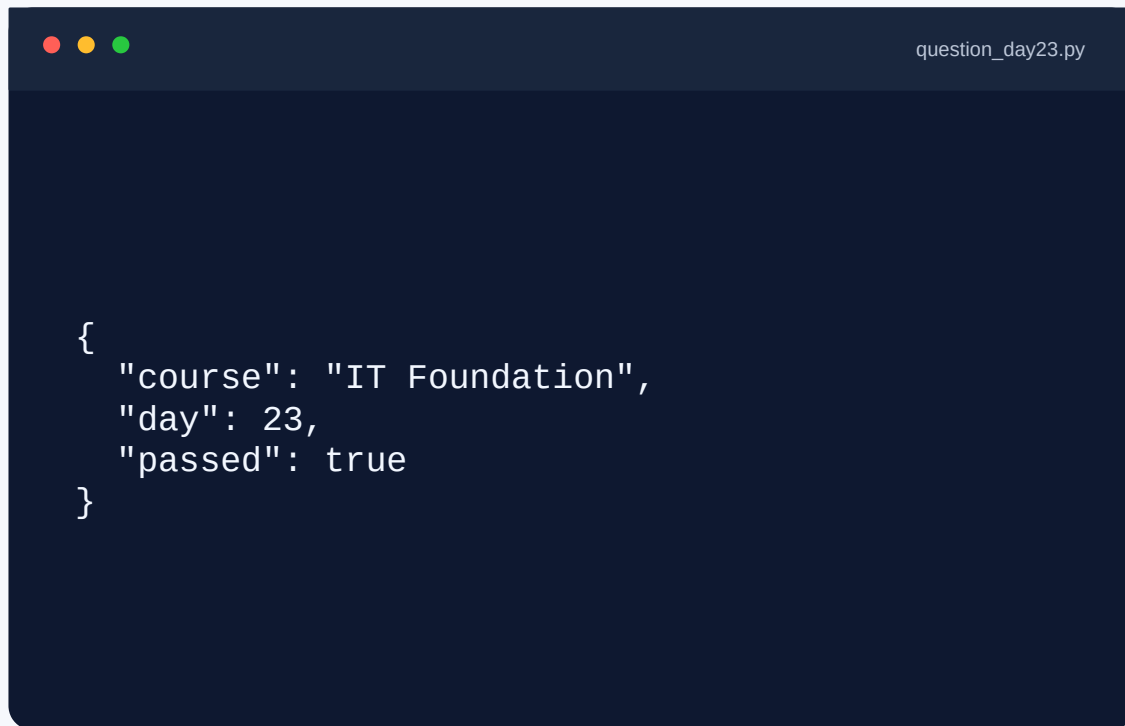


## Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

# Практика: что произойдет?

Ответь без запуска кода.



```
{  
    "course": "IT Foundation",  
    "day": 23,  
    "passed": true  
}
```

## Вопрос

Что программа сделает после запуска?

## Подсказка

Ищи последовательность: входные данные -> действие -> результат.

# Ответ и объяснение

Сравни со своим рассуждением.

## Ожидаемая логика

Этот пример относится к теме: API, REST и JSON. Связываем программы через понятный контракт.

## Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

## Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

# Практика: сделай руками

Небольшое задание по теме дня.

## Задание

Спроектируй endpoint: список курсов, курс, результат.

### Шаг 1

Создай простой файл или черновик.

### Шаг 2

Напиши минимальный рабочий вариант.

### Шаг 3

Запусти и проверь результат.

# Мини-тест

Выбери ответ и объясни почему.

## Вопрос

Какой HTTP-метод обычно используют для чтения ресурса без изменения?

**A**

POST

**B**

GET

**C**

DELETE

**D**

PATCH

# Ответ мини-теста

Проверяем не память, а понимание.

## Правильный ответ

В: GET

## Объяснение

GET предназначен для получения представления ресурса без его изменения.

# Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

## Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

## Как исправить

После каждого термина придумай свой пример и найди его в коде.

## Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

# Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

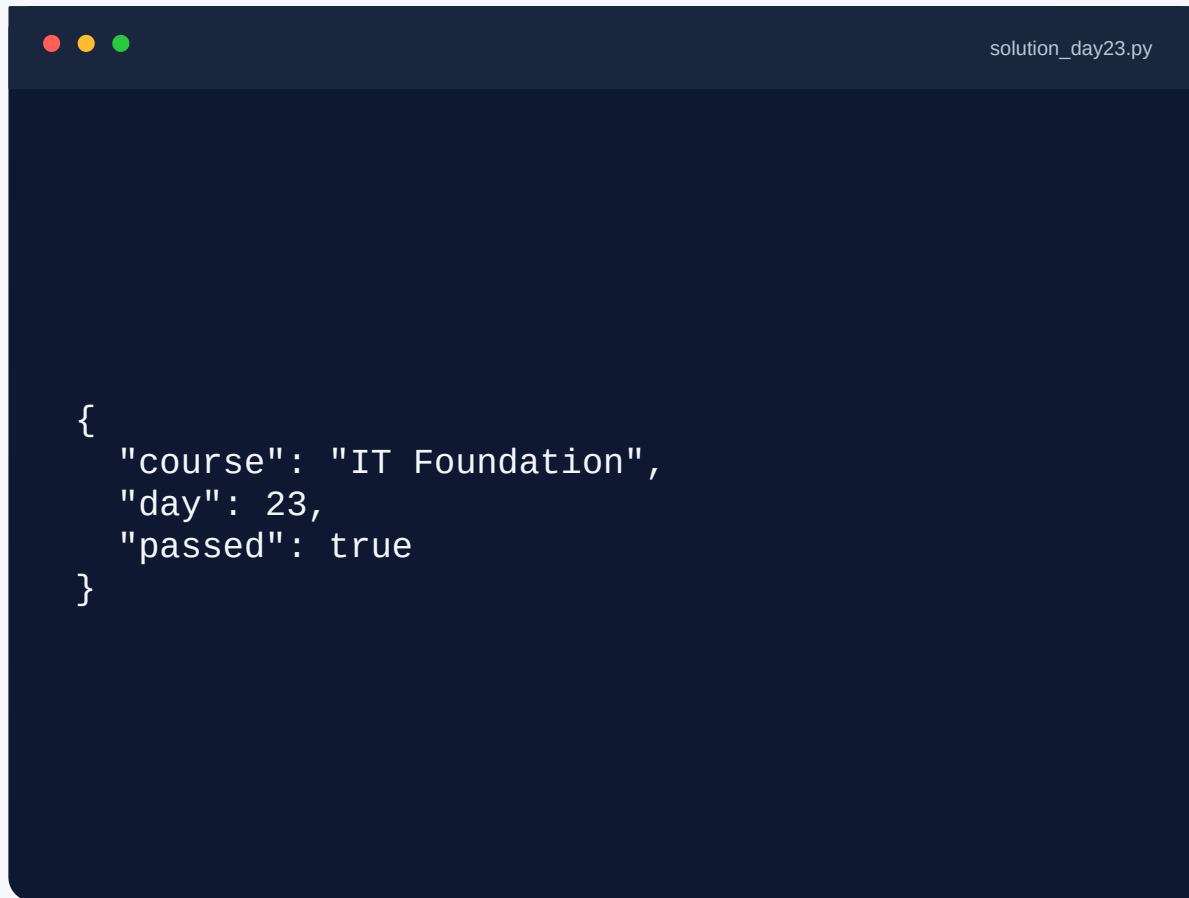
## Условие

Спроектируй endpoint: список курсов, курс, результат.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

# Разбор решения

После попытки сравни с этим подходом.



```
{  
  "course": "IT Foundation",  
  "day": 23,  
  "passed": true  
}
```

## Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

## Проверка

Запусти пример и объясни результат вслух.

# Задание со звездочкой

Немного сложнее, но без новой темы.

## Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

## Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

# Итог дня

Что должно остаться после урока.

- объяснять тему "API, REST и JSON" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

## Домашнее задание

Спроектируйте три endpoint для курсов: список, конкретный курс и сохранение результата. Укажите HTTP-метод каждого.